

Project Midterm
CS 220 Synthesis of Digital Systems
Spring 2026
University of California, Riverside

The goal of the project midterm is to establish the baseline design for your proposed optimizations. You will focus on the verification and synthesis of your selected design. Optimizations are optional for midterm. Below details the deliverables for the project midterm.

Baseline RTL

Code: All Verilog RTL files should be in a folder called *rtl*.

Report: A high-level explanation of each important design module. For any changes to the baseline RTL, document the changes made and justifications. Changes should only be fixing bugs and not adding new features.

Verification

Code: All verification and simulation files should be in a folder called *sim*.

There should be the main Verilog testbench that instantiates the top DUT module. It should also generate all the required stimulus signals. If external test vectors are used to supply the stimulus, you need to include the test vector generators.

You will need to write both random-constrained testcases and corner case testcases to verify your DUT. You should target to write 10 testcases for each type. These testcases should mainly check for the correctness of your design's functionalities. All testcases should be non-trivial and must pass (for both ideal and netlist simulations). You may use the same testbench for both ideal and netlist simulations, or use separate testbenches. The testcases can be the same for both.

Report: You will include a high-level description of the functionality of your testbench. If your netlist testbench is different, please describe how it is different from the ideal testbench. For each testcase you develop, include a brief summary of the verification goal and expected result. You may use a table to summarize these testcases.

Software Golden Model

Code: If your design requires a software golden model to verify its correctness and performance (AI accuracy, image fidelity etc.), these models (Python, C++ etc.) should be in a folder called *sw*. Any software testbench, signal generators, and/or software simulators you used should also be included.

Report: A high-level description of how your software golden model reflects the hardware implementation. Specify what metric(s) you choose to evaluate the software performance of your DUT.

Synthesis

Code: Your main synthesis script and any generated reports should be in a folder called *syn*. You may also put any other synthesis-related scripts here. You will use the synthesis flow to evaluate the frequency and area of your DUT. The design must be synthesized with worst-case corners:

- saed32rvt_ss0p75v125c.db
- saed32lvt_ss0p75v125c.db
- saed32hvt_ss0p75v125c.db

Note, you may mix and match LVT, RVT, and HVT cells to your design.

The priority is to close timing (no negative slacks) with the maximum frequency. Then with this frequency, derive the area and power (with PrimeTime). Please refer to the lecture for an explanation on what the maximum frequency means.

Report: Describe the main functionality of your synthesis script, any special synthesis methodology (top-down, incremental etc.), and how you determined the maximum frequency for your design. Summarize the final frequency (in MHz) and area (in μm^2 or mm^2) of your baseline design. For this frequency, report the timing slack in the critical path(s).

PrimeTime

Code: Your main PrimeTime script to measure the DUT netlist's power consumption and any generated reports should be in a folder called *ptpx*. The power should be measured by test vectors representative of real workloads for your DUT. The power should also be measured across an extended period of time (cycles) to reflect the average power consumption. You may optionally evaluate the peak power by reasonably stressing all of the DUT's inputs.

Report: Describe the main functionality of your ptpx script, any special power measurement methodology, and how you generated the input test vectors for your power measurement. Summarize the average (and optionally peak) power consumption (in mW or W) of your baseline design.

Midterm Report

- 1) Please refer to above on what to include for each section (RTL, verification etc.) in the report.
- 2) Validation of your reported PPA values and software metrics. You may validate against official documentations of these designs, and/or related designs in the literature.
- 3) A detailed plan on implementing your proposed optimizations. Preliminary results are optional.
- 4) A detailed plan on how to verify your proposed optimizations. If you use software models, explain how you plan to modify the software models to reflect your hardware changes.
- 5) A README section on how to execute your RTL simulation, hardware verification, synthesis, PrimeTime, and software verification to recreate reported results. The README should also explain any new packages, libraries, and/or virtual environment you introduced to accomplish the simulation, verification, and synthesis of your baseline.

Submission Instruction

- 1) For the code deliverables, place all the required folders (rtl, sim etc.) into a top-level folder called midterm-team-N, where N is your team number. Then place this midterm-team-N folder into a zip file. Make sure that any paths or directory variables are pointing to the correct locations within this midterm-team-N folder.
- 2) Midterm report.
- 3) Midterm presentation.

Grading

The project midterm is worth 10% of your total course grade. Each item is graded based on its completeness. Each complete item will receive full credit. An item with noticeably incomplete but reasonable results will receive partial credit. A missing item or item with entirely erroneous results will receive no credit.

Midterm presentation (2 pt)

Midterm report (total 4 pt):

- Write-up for individual sections (RTL, verification etc.) (1 pt)
- Validation of your reported results (1 pt)
- Design and verification plan for proposed optimizations (1 pt)
- README instruction (1 pt)

Code deliverables (total 4 pt):

- Baseline RTL and software golden model (1 pt)
- Verification testbench and testcases (1 pt)
- Synthesis scripts (1 pt)
- PrimeTime scripts (1 pt)